ASSIGNMENT 2 – CO2

TASK 07

Q7 - Improving Hospital Triage Using Priority Aging

Course: CSA0903 – Programming in Java

Problem Context: Railway e-Ticket Generation

Name: Sowmiya. D

Register Number: 192511063

7.1 Selected Limitation

The limitation selected from Task 06 is starvation in PriorityQueue.

In the original triage system, patients are selected according to their medical severity. Critical patients always receive priority over patients with lower severity.

If critical patients continuously arrive, patients with medium or low severity may wait for a very long time.

7.2 Problem Diagnosis

The original comparator uses only:

1. Medical severity.
2. Arrival order when severity is equal.

It does not consider how long a patient has been waiting.

Therefore, a patient with severity 5 may continue waiting while new severity 1 patients enter the queue.

7.3 Objective

To redesign the triage system so that patients who wait for a longer time gradually receive higher priority, while maintaining medical urgency.

7.4 Proposed Redesign – Priority Aging

Priority aging increases a patient's priority based on waiting time.

In this redesign:

- Every patient has an original severity.
- For every five waiting seconds, the patient's effective severity improves by one level.
- Severity cannot improve beyond level 1.

- If two patients have the same effective severity, the earlier arrival is selected.

Formula:

$$\text{Effective Severity} = \max(1,\text{Original Severity}-\lfloor\text{Waiting Time}/5\rfloor)$$

A lower effective severity means a higher treatment priority.

Example:

Patient	Original severity	Waiting time	Effective severity
Arun	5	0 seconds	5
Divya	3	5 seconds	2
Kumar	4	10 seconds	2
Priya	5	20 seconds	1

Priya's priority improves as she waits. This helps prevent starvation.

7.5 Implementation Approach

The redesigned system uses a PriorityQueue. Before selecting the next patient, it recalculates the effective severity of waiting patients and rebuilds the queue.

This is necessary because Java PriorityQueue does not automatically rearrange its elements when a value used by its comparator changes.

7.6 Validation

The redesign is tested with patients having different original severity levels and waiting times.

The expected result is that patients who have waited longer gradually move towards higher priority. Patients with the same effective severity are still selected according to arrival order.

7.7 Complexity Analysis

Operation	Complexity
Calculate effective severity	O(1) per patient
Rebuild the priority queue	O(n log n)
Peek next patient after rebuilding	O(1)
Remove selected patient	O(log n)

The redesigned system requires additional processing to rebuild the queue, but it provides improved fairness.

7.8 Conclusion

Priority aging improves the original hospital triage system by considering both medical severity and waiting time. It reduces the possibility of starvation while preserving priority-based treatment. The redesigned approach is more suitable when fairness and patient waiting time are important.

Java program:

```
import java.util.*;


public class Main {


   static class Patient {
      String name;
      int severity;
      int arrivalNumber;
      int arrivalTime;
      int effectiveSeverity;


      Patient(String name, int severity,
            int arrivalNumber, int arrivalTime) {


         this.name = name;
         this.severity = severity;
         this.arrivalNumber = arrivalNumber;
         this.arrivalTime = arrivalTime;
         this.effectiveSeverity = severity;
      }


      void updatePriority(int currentTime) {
```

```
        int waitingTime = currentTime - arrivalTime;
        int improvement = waitingTime / 5;

        effectiveSeverity =
                Math.max(1, severity - improvement);
    }

    @Override
    public String toString() {
        return name
                + " | Original Severity: " + severity
                + " | Effective Severity: " + effectiveSeverity
                + " | Waiting Time: " + (20 - arrivalTime)
                + " seconds";
    }
}

static final Comparator<Patient> COMPARATOR =
        Comparator.comparingInt(
                (Patient p) -> p.effectiveSeverity)
                .thenComparingInt(p -> p.arrivalNumber);

static PriorityQueue<Patient> refreshQueue(
        List<Patient> patients, int currentTime) {

    PriorityQueue<Patient> updatedQueue =
```

```
            new PriorityQueue<>(COMPARATOR);

    for (Patient patient : patients) {
        patient.updatePriority(currentTime);
        updatedQueue.offer(patient);
    }

    return updatedQueue;
}

static void demonstrateAging() {

    List<Patient> patients = new ArrayList<>();

    patients.add(new Patient("Arun", 5, 1, 0));
    patients.add(new Patient("Divya", 1, 2, 20));
    patients.add(new Patient("Kumar", 3, 3, 5));
    patients.add(new Patient("Priya", 4, 4, 10));

    int currentTime = 20;

    PriorityQueue<Patient> queue =
            refreshQueue(patients, currentTime);

    System.out.println("\nTRIAGE ORDER AFTER PRIORITY AGING");
    System.out.println("Current time: " + currentTime + " seconds");
```

```
    while (!queue.isEmpty()) {
        System.out.println(queue.poll());
    }
}

static void performanceTest(int size) {

    List<Patient> patients = new ArrayList<>();

    Random random = new Random(42);

    for (int i = 1; i <= size; i++) {

        int severity = random.nextInt(5) + 1;
        int arrivalTime = random.nextInt(20);

        patients.add(new Patient(
                "Patient-" + i,
                severity,
                i,
                arrivalTime
        ));
    }

    long start = System.nanoTime();

    PriorityQueue<Patient> queue =
```

```
            refreshQueue(patients, 20);

        long end = System.nanoTime();

        System.out.println("\nPatients: " + size);

        System.out.printf(
                "Queue rebuild time: %.3f ms%n",
                (end - start) / 1_000_000.0
        );

        System.out.println(
                "Next patient: " + queue.peek()
        );
    }

    public static void main(String[] args) {

        System.out.println("IMPROVED HOSPITAL TRIAGE SYSTEM");
        System.out.println("Priority Aging Interval: 5 seconds");

        demonstrateAging();

        System.out.println("\nPERFORMANCE TESTING");

        int[] sizes = {1000, 10000, 50000};
```

```
        for (int size : sizes) {
            performanceTest(size);
        }

        System.out.println("\nPriority aging tests completed.");
    }
}
```

Output:

IMPROVED HOSPITAL TRIAGE SYSTEM

Priority Aging Interval: 5 seconds

TRIAGE ORDER AFTER PRIORITY AGING

Current time: 20 seconds

Arun | Original Severity: 5 | Effective Severity: 1 | Waiting Time: 20 seconds

Kumar | Original Severity: 3 | Effective Severity: 1 | Waiting Time: 15 seconds

Divya | Original Severity: 1 | Effective Severity: 1 | Waiting Time: 0 seconds

Priya | Original Severity: 4 | Effective Severity: 2 | Waiting Time: 10 seconds

PERFORMANCE TESTING

Patients: 1000

Queue rebuild time: ... ms

Next patient: ...

... (10,000 and 50,000 patients)

Priority aging tests completed.

Implementation:

